2026-5-21 C++模板进阶

- 1. 非类型模板参数
- 2. 模板的特化
  - 2.1 概念
  - 2.2 函数模板特化
  - 2.3 类模板特化
    - 2.3.1 全特化
    - 2.3.2 偏特化
    - 2.3.3 类模板特化应用实例
- 3. 模板分离编译
  - 3.1 什么是分离编译
  - 3.2 模板的分离编译
- 4.总结
  - 4.1 优点
  - 4.2 缺点

1. 非类型模板参数

模板参数分类类型形参与非类型。

- 类型形参：出现在模板列表中，跟class或者typename之类的参数类型名称。
- 非类型形参：用一个常量作为类（函数）模板的一个参数，在类（函数）模板中可将该参数当常量来使用。
  实例代码如下：

```
//定义一个模板类型的静态数组
template<class T =int,size_t N=100>
class Stack {
private:
    T _a[N];
    int _top = 0;
    int _capacity = N;
};
int main() {
    Stack<int,10> s1;  //10
    Stack<int,1000> s2;//1000
    Stack<int> s3;     //100默认就是100
    Stack<> s4;
```

```
    //这里不支持
    /*int x;
    cin >> x;
    Stack<int, x> s5;*/

    cout << sizeof(s1) << endl;
    cout << sizeof(s2) << endl;
    return 0;
}
```

补充一个小知识：
下面的两个a1感觉不都一样吗？那array存在的意义是什么呢？

```
int main() {
    array<int, 10>a1;
    //int a1[10];
    a1[0] = 10;
    a1[9] = 100;
    //cout << a1[10] << endl;
    return 0;
}
```

`std::array` 相比原生数组，最大的优势在于它是容器，支持迭代器操作，且提供了 `.at()` 方法进行安全的边界检查（抛出异常）。虽然 `operator[]` 在某些调试模式下可能包含断言，但在 Release 模式下通常不进行检查以保证性能。
还有一点就是数组对于越界即使检查出来了，也只是限定写，不限定读。这种哪怕你读的那一块不在数组范围内，顶多读出来是乱码。而array的检查是很严苛的，既不可以读也不可以写。
使用的简单示例：

```
int main() {
    array<int, 10>a1;
    a1.fill(1);//fill（）将数组内所有元素都填充 成某个值
    for (auto e : a1) {
        cout << e << " ";
    }
    cout << endl;
    //还可以用array模拟二维数组
    array<array<int, 5>, 10>aa;
    return 0;
}
```

> 注意：
> 1.非类型模板参数只能是**整型常量、枚举、指针、引用**。C++20 之后允许浮点数，但不推荐。字符串字面量不允许。
> 2.非类型的模板参数必须在编译器就能确认结果。

2. 模板的特化

2.1 概念

通常情况下，使用模板可以实现与类型无关的代码，但对于一些特殊类型的可能会得到一些错误的结果，需要特殊处理。例如：
实现一个专门用于进行小于比较的函数模板：

```
template<class T>//1.先有一个基础的函数模板
bool Less(T left, T right) {
    return left < right;
}
int main()
{
    cout<< Less(1,2)<<endl;//结果正确
    Date d1(2022,7,7);
    Date d2(2022,7,8);
    cout<< Less(d1,d2)<<endl;//结果正确
    Date* p1 = new Date(2022,7,7);
    Date* p2 = new Date(2022,7,8);
    cout<< Less(p1,p2)<<endl;//结果错误
    return 0;
}
```

在上述例子中：最后的p1和p2比较的实际上是p1与p2的地址，无法达到预期结果。
模板特化:在原模板类的基础上，针对特殊类型所进行特殊化的实现方式
这个时候就要对模板进行特化，即在原模板类的基础上，针对特殊类型所进行的特殊化实现方式。模板特化可以分为**函数模板特化**与**类模板特化。**

2.2 函数模板特化

函数模板的特化步骤：

1. 必须要现有一个基础的函数模板
2. 关键字template后面接一对空的尖括号<>

3. 函数名后面跟一对尖括号，尖括号中需要指定特化的类型，在调用的时候会自动的匹配类型，有现成的就用用现成的。匹配原则。

```
//函数模板--参数匹配
template<class T>//1.先有一个基础的函数模板
bool Less(T left, T right) {
    return left < right;
}
//特化
template<>//关键字后面加一个<>
bool Less<double*>(double* left, double* right) {//函数名后面加一个<>里面跟类型
    return *left < *right;
}
//特化
template<>//关键字后面加一个<>
bool Less<string*>(string* left, string* right) {//函数名后面加一个<>里面跟类型
    return *left < *right;
}
//特化和函数模板可以同时存在
int main() {
    double* p1 = new double(2.2);
    double* p2 = new double(1.1);
    cout << Less(p1, p2) << endl;
    //如果你的模板参数不特化直接比就会出现问题
    //根本原因在于你这里实际上比的是p1和p2的地址，哪怕p2的地址后分配，也不能保证说p2的地址小于p1
    //这里要比较的话是比较p1与p2解引用之后的内容，所以我们就有了模板的特化

    string* p3 = new string("111");//这里如果不写特化的话会报错不匹配无法初始化为double*
    string* p4 = new string("222");
    cout << Less(p3, p4) << endl;
    return 0;
}
```

4. 函数形参表：必须要和模板函数的基础参数完全相同，如果不同编译器可能会报一些奇怪的错误
   例如：

```
//函数模板--参数匹配
template<class T>//1.先有一个基础的函数模板
bool Less(T left, T right) {
    return left < right;
}
```

```
//加const的特化，简单回顾一下
//1.const 在*的左边是指针指向的对象不能修改（说人话：指针指向的值不能修改）
//2.const 在*的右边是指针本身不能修改（说人话：指针的指向不能修改）
template<>//这里如果要加const限制模板参数的话，应该限制的是值不能修改
//错误写法,这里的本质原因是：模板特化的参数类型必须和模板实参严格一致
//bool Less<double*>(const double* left,const double* right) {//函数名后面加一个<>里面跟类型
//  return *left < *right;
//}
bool Less<double*>(double* const left, double* const right) {//函数名后面加一个<>里面跟类型
    return *left < *right;
}
int main() {
    double* p1 = new double(2.2);
    double* p2 = new double(1.1);
    cout << Less(p1, p2) << endl;
    return 0;
}
```

注意，一般情况下如果函数模板遇到不能处理或者处理有误的类型，为了实现简单通常都是将该函数直接给出。

```
bool Less(Data* left,Data right) {
    return *left < *right;
}
```

这种实现简单明了，代码可读性高，容易书写，因为对于一些参数类型复杂的函数模板，特化时特别给出，因此函数模板不建议特化。

2.3 类模板特化

2.3.1 全特化

全特化即是将模板参数列表中所有的参数都确定化,精准特化。

```
template<class T1,class T2>
class Data {
public:
    Data(){
        cout << "Data<T1,T2>" << endl;
```

```
        }
    void f1(){}
private:
    T1 _d1;
    T2 _d2;
};

//全特化
template<>
class Data<int, char> {
public:
    Date() {
        cout << "Data<int,char>" << endl;
    }
};
```

2.3.2 偏特化

任何针对模板参数进行进一步条件限制的特化版本，特化所有类型

- 部分特化
  将模板参数表中的一部分参数特化。

```
template<class T1,class T2>
class Data {
public:
    Data(){
        cout << "Data<T1,T2>" << endl;
        }
    void f1(){}
private:
    T1 _d1;
    T2 _d2;
};

//偏特化：特化部分参数
template<class T1>
class Data<T1, char> {
public:
    Data() {
        cout << "Data<T1,char>" << endl;
    }
};
```

- 参数更新进一步的限制
  偏特化并不仅仅是指特化部分参数，而是针对模板参数更进一步的条件所设计出来的一个特化版本。

```
//两个参数偏特化为指针类型
template<class T1, class T2>
class Data<T1*,T2*>{//所有指针都过来
public:
    Date() {
        cout << "Data<T1*,T2*>" << endl;
    }
    void f1() {
        T1 x1;
        cout << "type:"<<typeid(x1).name() << endl;
        T1 x2;
        cout << "type:"<<typeid(x2).name() << endl;
    }
};
//两个参数偏特化为引用类型
template<class T1, class T2>
class Data<T1&, T2&> {
public:
    Data() {
        cout << "Data<T1&,T2&>" << endl;
    }
    void f1() {
        T1 x1;
        cout << "type:" << typeid(x1).name() << endl;
        T1 x2;
        cout << "type:" << typeid(x2).name() << endl;
    }
};
//两个参数偏特化一个为指针类型一个为引用类型
template<class T1, class T2>
class Data<T1*, T2&> {
public:
    Data() {
        cout << "Data<T1*,T2&>" << endl;
    }
    void f1() {
        T1 x1;
        cout << "type:" << typeid(x1).name() << endl;
        T1 x2;
        cout << "type:" << typeid(x2).name() << endl;
```

```
    }
};
```

偏特化并不仅仅是指特化部分参数，而是针对模板参数更进一步的条件限制所设计出来的一个特化版本。
全特化和偏特化二者可以同时存在，但是参数匹配的情况下优先选全特化。但一般：**精确匹配 > 偏特化 > 基础模板**。

2.3.3 类模板特化应用实例

我们之前写的优先队列除了用仿函数实现，还可以用类模板的特化实现：

```
template<class T>
class Less {
public:
    bool operator()(const T& x, const T& y) {
        return x < y;
    }
};
//前置声明
class Data;
template<>
class Less<Data*> {
public:
    bool operator()(Data* const x, Data* const y) {
        return *x < *y;
    }
};
template<>
class Less<int*> {
public:
    bool operator()(int* const x, int* const y) {
        return *x < *y;
    }
};
```

在.cpp中：

```
#include"priority_queue.h"//放到这里让其可以找到Date，不能只能在重新写一个.h和.cpp文件在.h
//文件中调用
```

```
//在.h文件中不宜使用前置声明，适用于只用这个类，不用这个类里的东西
    int main() {
        //显示实现仿函数的控制比较逻辑
        //ZL::priority_queue<Date*, vector<Data*>, PDateLess> q1;
        //缺省仿函数类，针对Data*进行特化
        ZL::priority_queue<Data*> q1;
        q1.push(new Date(2018, 10, 29));
        q1.push(new Date(2018, 10, 28));
        q1.push(new Date(2018, 10, 30));
        while (!q1.empty()) {
            cout << *q1.top() << " ";
            q1.pop();
        }
        cout << endl;
        //下面也可以特化
        ZL::priority_queue<int*> q2;
        q2.push(new int(3));
        q2.push(new int(1));
        q2.push(new int(2));
        while (!q2.empty()) {
            cout << *q2.top() << " ";
            q2.pop();
        }
        cout << endl;
        //其他指针都按照指向的对象比较
        //char*按照指针比较
        //全特化的类型更加确定，偏特化还是需要实例化的，都存在优先走全特化
        ZL::priority_queue<char*> q3;
        q3.push(new char('a'));
        q3.push(new char('b'));
        q3.push(new char('c'));
        while (!q3.empty()) {
            cout << *q3.top() << " ";
            q3.pop();
        }
        cout << endl;
        return 0;
}
```

3. 模板分离编译

3.1 什么是分离编译

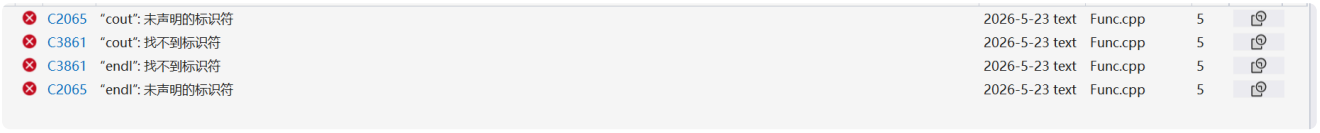
为什么学到模板的时候不说声明与定义分离：
一个程序（项目）由若干个源文件共同实现，而每个源文件单独编译生成目标文件，最后将所有目标文件链接起来形成单一的可执行文件的过程称为分离编译模式。

3.2 模板的分离编译

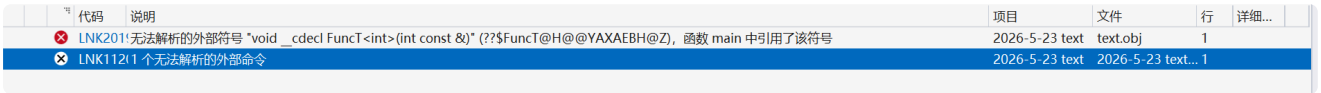
模板的声明与定义分离开，在头文件中进行声明，源文件中完成定义。

```
//Func.h
#pragma once
//这里的问题是这个cpp里面既找不到iostream,也没有std::的展开
template<class T>
void FuncT(const T& x);
//Func.cpp
#include"Func.h"
template<class T>
void FuncT(const T& x) {
    cout << "void FuncT(const T& x)" << endl;
}
//text.cpp
int main() {
    //如果不做声明和定义分离
    FuncT(1);
    return 0;
}
```

函数模板,声明与定义后编译器就不认识cout了，这是为什么？



这里向上查找的的时候没有iostream，std::会出问题。需要再在.h文件中包含iostream头文件。



现在我们又出现了链接错误，什么时候会出现链接错误呢？我们用普通函数的声明和定义分离来类比。普通函数声明和定义分开时，如果你没有定义的话程序就会发生链接错误，编译器它找不到具体的实现。模板有定义也会发生错误，这是为什么呢？
我们首先理解编译器对这几个文件的编译机制是什么？
Func.h、Func.cpp、text.cpp 这三个文件的处理机制如下：

- **预处理**：展开头文件、宏替换、条件编译、去掉注释 → 生成 **Func.i, text.i** 文件（预处理后的源码）
- **编译**：检查语法，将代码翻译成汇编语言 → 生成 **Func.s, text.s** 文件（汇编代码）
- **汇编**：将汇编代码转换为机器能识别的二进制指令 → 生成 **Func.o, text.o** 文件（目标文件/二进制机器码）
- **链接**：合并所有目标文件，解析符号引用（如函数地址），最终生成可执行程序（如 a.out 或 .exe）
  先用函数调用来理解编译的过程：函数调用的底层指令是call地址，编译阶段没有地址，因为这个时候只有声明，只有声明就没有地址。函数地址就是一个问号，声明是一种承诺，我承认这种承诺编译会通过，传一个参数会就编译会报错，这个就是检查匹配的。什么时候去找地址，链接的时候，找到地址就是真的，没有的话链接错误。
  那函数模板有定义为什么会错？为什么找不到它的地址？函数模板要实例化了才会分配具体的空间拥有地址，项目在链接的时候都是单独交互的，如果编译器不知道实例化成什么，模板在编译时就找不到到底要编译成什么类型的，所以就没有地址。
  既然是实例化的问题，那解决方案就有：

1. 在定义的时候显示实例化，在cpp文件里告诉编译器这个模板到底是个什么类型。但是就失去了模板的味道了，就感觉很多余了，模板只建议声明和定义到同一个文件

```
//显示实例化
template//告诉其是模板的显示实例化
void FuncT(const int& x);
```

2. 直接定义到.h文件中也不会出现上述问题，因为预处理的时候声明和定义都过来了，调用的话就有了定义，编译时实例化生成了函数的地址，不需要链接。

```
template<class T>
void FuncT(const T& x) {
    cout << "void FuncT(const T& x)" << endl;
}
```

类模板也相同：

```
template<class T>
class Stack {
public:
    void Push(const T& x);
};
template<class T>//和上面的定义一起定义在同一个文件中
```

```
void Stack<T>::Push(const T& x) {//类模板定义也是类里面函数的定义
    cout << "void Push(const T& x)" << endl;//这个单独写在一个.cpp文件中也会报错
的
}
```

有的模板文件的后缀是.hpp

4.总结

4.1 优点

1. 模板复用了代码，节省资源，更快的迭代开发，C++的标准模板库因此而产生
2. 增强了代码的灵活性

4.2 缺点

1. 模板会导致代码膨胀的问题，也会导致编译的时间变长
2. 出现模板编译错误时，错误信息报的往往没那么准确，不易定位错误